O'REILLY

6th Edition

Learning Python

Powerful Object-Oriented Programming

Mark Lutz

Learning Python

SIXTH EDITION

Powerful Object-Oriented Programming

Mark Lutz

Learning Python

by Mark Lutz

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Louise Corrigan

Development Editor: Sara Hunter

Production Editor: Kristen Brown

Copyeditor: nSight, Inc.

Proofreader: Piper Content Partners

Indexer: nSight, Inc.

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

March 2025: Sixth Edition

Revision History for the Sixth Edition

- 2025-02-25: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098171308* for release

details.

978-1-098-17130-8

[LSI]

[Dedication]

To Vera.

You are my life.

Preface

If you're browsing a bookstore and trying to make sense of this book, try this:

- *Python* is one of the most widely used programming languages in the world. It's part of nearly every role that computers play in our lives, and its relative ease of use makes it an ideal way to get started with programming.
- *This book* is a tutorial that teaches Python language fundamentals in depth. Its content is aimed at Python newcomers of all stripes, applies to every role that Python plays, and is based on decades of feedback from real learners like you.
- *This edition* updates this book for a decade of changes in Python and its world. It drops coverage of the now-defunct Python 2.X, explores new tools added to Python through version 3.12, and applies to other Pythons past and future.

The rest of this preface provides more background info on this book and its subject. It explains what's changed since the prior edition, debuts the book's examples package, and may help you get oriented before jumping into details.

Python

By most metrics you'll find on the web today, Python is now either the most-used programming language on the planet or very near the top of the list. The oft-cited TIOBE popularity index, for example, has ranked Python most popular for several years. As this edition is being written in 2024, it lists Python at #1 and well ahead of its nearest followers, C, C++, and Java.

Popularity ranks are prone to change, of course, and rely on usage metrics that are open to debate that we'll skip here. Based on every signal available, though, the Python revolution has clearly happened. It's now a ubiquitous, *go-to language* in web development, scientific programming, systems administration,

AI, and nearly everything else computers do today. Thanks to its relative simplicity, Python is also commonly used to introduce newcomers to computer science across the education spectrum.

In fact, it's now fairly safe to say that Python played a pivotal role in *changing the world*. By spearheading a shift from statically typed compiled languages to dynamically typed scripting languages, Python ushered in changes that were at least as profound as those of the earlier transition from machine language to compiled languages. The scripting-language shift both enabled tasks formerly impractical, and opened the field to nonprofessional contributors. In the process, it propelled computers to a prominence that would have been unthinkable in decades past. The internet, for example, simply could not be what it is today without tools like Python. For better and worse, Python enables the new.

Lofty goals aside, all of this has two tangible implications for this book. First, because this is now a post-revolution Python world, this edition does much less *cheerleading* than its predecessors. There's no reason to waste your time promoting a tool that's already arrived. This edition still summarizes Python's value proposition in Chapter 1, but the domains and tools that you're likely to explore after learning the basics here are readily available on the web and change too regularly to cover in a fundamentals book like this in any event.

Second, Python's popularity means that by reading this book, you'll be adding a *valuable skill* to your toolset, which will help you in a wide variety of computer-software tasks. Learning Python will both make entire domains accessible to you and enable you to achieve programming goals that might otherwise be difficult or impossible. Python users now enjoy a wealth of prior art ready to be leveraged in a language that accelerates their work.

That said, Python isn't the only game out there, and you'd also be well served by learning computer science from the ground up—the *full stack* in developer speak. Studying lower-level languages like C and Java, for example, can still give you a much more complete perspective than scripting languages alone and help you solve complex problems as they arise. Python itself, after all, is just a C program in its most-used flavor.

Even so, Python is a great place to start, and enough for many a task. While it's not without warts and suffers from the thrashing that's endemic to software today, a multitude of developers still find Python a lot more fun to use than other

tools. Java and C++, for example, seem languages designed for middle management: they hobble programmers with training wheels and bureaucratic hurdles that have little to do with your program's goals. Python, in sharp contrast, remains *more ally than obstacle*. That viewpoint is naturally subjective, but you've come to the right place to do the math on this yourself.

This Book

This book is a tutorial on the Python language and a classic in its domain. It's the product of *three decades* spent using, promoting, and teaching Python, and dates back to the mid-1990s, when Python was still at version 1.X, and the web was just something developers mused about over lunch. Although the focus here is firmly on the present, that legacy naturally adds some historical context that will help you understand Python more deeply. Despite what you may have heard, the past matters, especially in knowledge-based fields.

Just as importantly, this book has always been based on live-and-in-person *feedback* from Python beginners struggling to learn Python for the first time. This feedback mostly owes to Python training classes taught over a period of two decades. While these classes have now gone the way of the dodo and Yahoo, this book takes care to retain its learner-inspired material because that's much—if not most—of its value.

As a result, if you're like most of the thousands of learners whose experiences have been captured here, you'll probably find that this book works like a self-paced version of the Python training sessions from which it arose. You may sometimes even find that it answers your questions before they are asked because a host of learners before you have had the same queries. This isn't clairvoyance; it simply reflects the fact that learning resources do best when they listen to learners.

It's also worth noting up front that this book sometimes *critiques* Python changes while presenting them. Critical thinking is crucial in engineering domains—especially in one caught up in an arms race that convolutes tools used by millions of people. On some levels, Python remains a constantly morphing sandbox of ideas that too often prioritizes changer hubris over user need, and this book is not shy about calling this out. That said, the main goal here is to educate,

not criticize, and opinions are always, well, opinionated. Although views here reflect decades of using and teaching Python, you should always judge the net worth of Python changes for yourself in whatever world you've been cast.

This Edition

This edition completely drops coverage of *Python 2.X*, the earlier version of the language, and adds new coverage of recent changes in *Python 3.X*, the newer and incompatible version. When the prior edition was published in 2013, Python 2.X was still widely used and probably even dominant. Because of that, the prior edition had to cover both the established 2.X and the new and upcoming 3.X, which at times made for a twisted tale indeed.

Over a decade later, 2.X has been officially sunsetted, and the Python world has adopted 3.X so fully that 2.X constitutes an unwarranted distraction to today's Python learners. Hence, after a decades-long tenure, 2.X-specific content has been cut here to make room for new 3.X topics and address book size in general. Formally, this edition has been updated to be current with *Python 3.12* and its era, though it also previews 3.13 mods, and its focus on fundamentals makes it generally applicable to both older and newer Python versions.

Before the emails start flooding in, this book wants to make clear that it regrets the loss of historical context (and secretly pines for the simpler days of 2.X too). But 3.X is a substantial topic all by itself, without bifurcating the story and increasing the page count for a Python version that is now little used. So go well into that good Python night, 2.X, and long live 3.X. Unless stated otherwise, "Python" in this edition simply means the 3.X line in general and 3.12 and later in particular.

In terms of *3.X mods*, this edition newly covers `f'…'` f-string literals, `:=` named-assignment expressions, `match` statements, type hinting, `async` coroutines, star-unpacking proliferation, underscore digit separators, *__main__.py* package files, `__getattr__` module hooks, `except*` exception groups, dictionary-key insertion order, positional-only function arguments, hash-based bytecode files, and other additions, deprecations, and mutations that have cropped up over the last decade plus.

Among these, type hinting and `async` coroutines are not covered in depth—by design. The former is an optional and academic tool wholly unused by Python itself and at odds with its core principles. The latter is an advanced applications tool and has morphed constantly since its inception. And both quickly head over complexity cliffs that push them out of scope for Python beginners. When needed, supplemental info on such narrow topics is always just a search away on the web. Here, the goal is learning to walk well before trying to run.

Among other noteworthy changes this time around:

- The *Unicode* content in the advanced part's Chapter 37 is new and improved because this topic is now an essential in Python 3.X and the world at large.
- Usage coverage, including the new Appendix A, gives more focus to *macOS*, *Android*, *Linux*, and *iOS* because not all of this book's readers use Windows.
- Most code-file examples now have numbered *captions* because the extra formality distinguishes them better in the book, and it's worth the space.
- Some *redundancy* has been trimmed, but not all, because repetition is useful and even important in learning resources.
- The *size* of this book was reduced by the prior bullet, rewrites and flow mods, and the net of 2.X cuts and 3.X inserts because it's less to grok.
- The size of the *print* version of this book was further reduced by moving two advanced but optional chapters online (Chapters 38 and 39) because it's less to lug.
- Fictitious *names* in examples are more gender neutral: "Bob" is now an ambiguous "Pat" unless paired with "Sue" as before because it better defuses bias.
- The *Monty Python* references have been dropped because they can be confusing and might be divisive, and borrowing personality from media seems cheap.

- Both *first-person voice* and *personal anecdotes* have been globally sacked because you've bought this book to learn Python, not an author's life story.

About the last two: Python's namesake was funny stuff, to be sure, but compulsively aping the work of a nearly all-male comedy group can seem like the secret handshake of an exclusive boys' club in hindsight. And while an occasional "I" or "my" might add color or credibility, overuse tends to come off as narcissism. Hence, the former 1k "spam" are now symbols more inclusive, and this book's three-decade tenure will have to speak for itself.

Despite all the mods, this edition remains much more *technical novel* than reference manual, and meaty enough to be comparable to a *full-semester class* on Python and programming. It introduces topics and expands them in later chapters as recurring themes, accumulating comprehensive coverage along the way. There are Python quick-reference resources at *python.org* and a multitude of blogs and videos that promise to teach Python rapidly. This book is for those who know that learning something well requires a bit more.

Media Choices

As of this writing, this book is destined to be available in three forms: *print* (i.e., paper), *ebook* (e.g., PDF, ePub, and Kindle), and *online* (a.k.a. web). The latter means the publisher's subscription service, currently branded as the O'Reilly learning platform (f.k.a. Safari). Naturally, each medium has valid uses that vary per reader. For instance, many prefer print for linear reads and electronic media for random searches and code copy/paste.

You're welcome to use the forms that work best for you, of course, but should carefully weigh the inherent *privacy trade-offs* of online media. By now, it should be abundantly clear that online anything comes with a potential for covert use and sale of customer information and access, which print and ebook media largely avoid. While monetization schemes vary, online users just might have a legion of salespeople peering over their shoulders as they use products for which they've already paid in full.

So please be careful out there. Unless you must use an employer's online

subscription, this book suggests vetting your media options wisely and generally recommends its *print and ebook* forms to protect your privacy whenever possible. Your life really shouldn't be turned into a revenue stream unless you get reimbursed for it.

One last media tip: this book may also be fed by its publisher into *generative AI* models, the current hot topic in the tech press. Although this may prove useful for looking up isolated facts, it's not deep learning and isn't necessarily any more reliable than the least of the gossip it regurgitates. Until the world figures this out, please use wisely.

Updates and Examples

As most authors would attest, it's shockingly easy to miss typos in material you've read hundreds of times, and incompatible change is a norm in computer book topics. Hence, this edition, like all its predecessors, expects to be updated regularly after its publication.

This book's supplements, example files, and clarifications and corrections (a.k.a. errata) will all be maintained on the web. Here are the main coordinates for these online resources; as usual, consult your local search engine if these change over time:

Author website

> This site will be used to post general *notes and updates* related to this text or Python itself—a hedge against future changes and a sort of virtual appendix to this book.

Book examples

> This site will host this book's *examples package*, with both individual files to view and save online and a ZIP file of all the examples to download to your device.

Publisher website

> This site will maintain this edition's *errata list*, and chronicle patches applied

to the text in reprints. It will also link to other formats, including ebooks.

The second of these is home to the examples' *source code*. Please see the usage notes there, as well as the examples' coverage in "Where to Run: Code Folders". There is no plan to host the examples on *GitHub* today, because that site's learning curve is a lot to ask of beginners, and its commercial agendas should be cause for concern.

At any of the preceding websites, all *error reports* and *suggestions* for this book are welcome, and this feedback is invaluable for book quality. But please keep it fact-based and civil. Posts on the errata list have been mostly constructive, but the list has limited utility, and has been known to attract the usual trolls. Such is life in the age of global conversation.

Conventions and Reuse

This book's mechanics will make more sense once you start reading it, but as a reference, this book uses the following typographical conventions:

Italic

Used for email addresses, URLs, filenames, pathnames, and emphasizing new terms when they are first introduced

`Constant width`

Used for program code, the contents of files and the output from commands, and to designate modules, methods, statements, and system commands

`Constant width bold`

Used in code sections to show commands or text that would be typed by the user, and, occasionally, to highlight portions of code

`Constant width italic`

Used for replaceables (content you must fill in) and some comments in code sections

> **NOTE**
>
> This element indicates a tip, suggestion, or general note relating to the nearby text. The icon may make more sense if you imagine a crow sounding an alarm.

Three more quick content notes here: first, you'll find occasional *sidebars* (delimited by boxes) and *footnotes* throughout, which are often optional reading but provide additional context on the topics being presented. For instance, the sidebars that begin with "Why You Will Care:" amplify language topics with real-world use cases.

Second, Python *error messages* are often shortened in this book to conserve space. Please run offending code on your own device for the full text. Some messages include stack traces, and some have sprouted location indicators and speculative "Did you..?" help in the latest Python that might be useful for beginners, though veterans' mileage may vary; either way, the extra text is excessive in a book tight on space.

Finally, the publisher maintains a standard statement about reusing code in this book, though the short, interactive code snippets used broadly here are hardly worth the legalese. Please see the book websites described earlier for the formal reuse story if you must care.

Acknowledgments

In keeping with the depersonalization goal discussed earlier, this edition will forego the usual lengthy acknowledgments of its predecessors. Instead, it extends simple gratitude to the scores of former students and readers, who largely shaped this book; the publishing company, which enabled this book to both reach learners and improve with time; the host of users, contributors, and promoters, who made Python what it is today; and the subject of this book's dedication page, who patiently tolerated yet another book project. This one might be the last, but you never know what a bored author might next do.

Part I. Getting Started

Chapter 1. A Python Q&A Session

If you're reading this book, you may already know what Python is and why it's an important tool to learn. If you don't, you probably won't be sold on Python until you've learned the language by reading the rest of this book and have done a project or two. But before we jump into details, this first chapter will briefly introduce some of the main reasons behind Python's popularity and begin sculpting a definition of the language. This takes the form of a question-and-answer session, which addresses some of the most common queries posed by beginners—like you.

Why Do People Use Python?

Because there are many programming languages to choose from, this is the usual first question of newcomers and a great place to start. Given that millions of people use Python today, there really is no way to answer this question with complete accuracy; the choice of development tools is often based on unique constraints or personal preference.

But after teaching Python to hundreds of groups and thousands of students, some common themes have emerged. The primary factors cited by Python users seem to be these:

Software quality

> For many, Python's focus on readability, coherence, and software quality in general sets it apart from other tools in the programing world. Python code is designed to be *readable*, and hence reusable and maintainable—much more so than traditional scripting languages. The uniformity of Python code makes it relatively easy to understand, even if you did not write it. In addition, Python has deep support for more advanced *software reuse* mechanisms, such as object-oriented (OO) and functional programming, that can further

promote code quality.

Developer productivity

Python boosts developer productivity many times beyond compiled or statically typed languages. As one measure of this, Python code is typically *one-third to one-fifth* the size of equivalent C++ or Java code. That means there is less to type, less to debug, and less to maintain after the fact. Most Python programs also run immediately, without the lengthy compile and link steps required by some other tools, further boosting programmer speed.

Program portability

Python programs generally run unchanged on *all major computer platforms*. Porting a Python program between Linux and Windows, for example, is often just a matter of copying its code between machines. Moreover, Python offers multiple options for coding portable graphical user interfaces, database access programs, web-based systems, and more. Even operating system interfaces as proprietary as program launches and directory processing are as portable in Python as they can possibly be.

Application support

Python comes with a large collection of prebuilt and portable functionality, known as the *standard library*. This library supports an array of application-level programming tasks, from text pattern matching to network scripting. In addition, Python can be extended with both homegrown libraries and a vast collection of third-party software. As covered ahead, Python's *third-party domain* offers tools for website construction, numeric programming, AI, and much more. The NumPy extension, for instance, has elevated Python to a core tool in science, technology, engineering, and math (*STEM*), and Django

and PyTorch have done similar for the web and AI.

Component integration

Python scripts can communicate with other parts of an application using a variety of integration mechanisms. Such integrations allow Python to be used as a product *customization and extension* tool. For instance, Python code can invoke compiled libraries and be run by compiled programs, interact with other components over networks, and use Android and iOS toolkits on smartphones. In fact, many Python core tools, including files, ultimately use precoded interfaces to system libraries; even if your program is all Python, it's not standalone.

Love of craft

Because of Python's ease of use and built-in toolset, it can make the act of programming *more pleasure than chore*. Although this benefit is intangible and subjective, its effect on productivity is an important asset. People do get paid for coding Python, but many use it just for *fun*—a testimonial rare in the software field.

Of these factors, the first two—quality and productivity—are probably the most compelling benefits to most Python users; let's take a closer look at each.

Software Quality

By design, Python has a deliberately simple and readable syntax and a highly consistent programming model. As a slogan at an early Python conference attested, the net result is that Python seems to *fit your brain*—that is, features of the language interact in consistent and limited ways and follow naturally from a small set of core concepts. This makes the language easier to learn, understand, and remember. In practice, Python programmers do not need to constantly refer to manuals when reading or writing code; it generally just makes sense.

By philosophy, Python adopts a somewhat minimalist mindset. This means that although there are usually multiple ways to accomplish a coding task, there is usually just one obvious way, a few less obvious alternatives, and a small set of coherent interactions throughout. Moreover, Python usually doesn't make arbitrary decisions for you; when interactions are ambiguous, explicit intervention is preferred over "magic." In the Python way of thinking, explicit is better than implicit, and simple is better than complex.

Beyond such design themes, Python includes tools such as modules and object-oriented programming (OOP) that naturally promote code reusability in skilled hands of the sort you're about to acquire. And because Python is focused on quality, most Python programmers naturally are too; this can be a crucial advantage when it's time to use someone else's Python code.

Developer Productivity

If you've worked in the software field, you know that it can be a dynamic and bumpy ride. During the great internet boom of the mid-to-late 1990s, it was difficult to find enough programmers to implement software projects; developers were asked to implement systems as fast as the internet evolved. In later eras of economic recession and layoffs, the picture shifted; programming staffs were often asked to accomplish the same tasks with even fewer people.

In both of these scenarios, Python has shined as a tool that allows programmers to get more done with less effort. It is explicitly optimized for *speed of development*—its simple syntax, dynamic typing, lack of compile steps, and built-in toolset allow programmers to develop programs in a fraction of the time needed when using some other tools.

The net effect is that Python typically increases developer productivity many times beyond the levels supported by traditional languages and often makes the impossible possible. That's good news in both boom and bust times, and everywhere the software industry goes in between.

Is Python a "Scripting Language"?

Maybe. Python is often applied in scripting roles, but not always. It's regularly

called an *object-oriented scripting language*—a definition that blends support for OOP with an orientation toward scripting contexts, but this may be too narrow and dismissive. If pressed for a one-liner, Python is probably better known as:

> *A general-purpose programming language that blends procedural, functional, and object-oriented paradigms and accelerates software development by reducing complexity.*

That may not fit on a t-shirt quite as well, but it captures both the richness and scope of today's Python.

Nevertheless, the term *scripting* seems to have stuck to Python like glue, perhaps as a contrast with the larger efforts required by some other tools. For example, some people use the word "script" instead of "program" to describe a Python code file, because it seems simpler and less formal to code. More usefully, others reserve "script" for a top-level file, and "program" for a more sophisticated multifile application, both of which are common in Python.

Because the term *scripting language* has so many different meanings to different observers, though, some would prefer that it not be applied to Python at all. In fact, people tend to make three very different assumptions when they hear Python labeled as such, some of which are more useful than others:

Shell tools

Sometimes when people hear Python described as a scripting language, they think it means that Python is a tool for coding operating-system-oriented scripts. Such programs are often launched from console command lines and perform tasks such as processing text files and launching other programs.

As you'll learn ahead, Python programs can and do serve such roles, but this is just one of dozens of common Python application domains. It is not just a better shell-script language.

Control language

To others, scripting refers to a "glue" layer used to control and direct (i.e., script) other application components. As noted earlier, Python programs are indeed often deployed in the context of larger applications. For instance, to